

Inteligencia artificial avanzada para la ciencia de datos II

Fidel Alexander Bonilla Montlavo

Instituto Tecnológico y de Estudios Superiores de Monterrey

A01798199

Querétaro, México

9 de septiembre de 2025

1. Dastate:

Objetivo: Clasificar el contenido emocional de un texto corto en una de 5 categorías (label = 0–4).

Tamaño: 416,810 registros y 1 atributo de entrada tipo texto y 1 variable objetivo siendo numérica.

1.1 Contenido del Dataset:

1.1.1 Información de la emoción:

1. text (*string*)
Texto libre expresando un pensamiento o emoción

1.1.4 Variable objetivo

2. label (*entero entre 0 y 4*)
Categoría emocional asociada al texto

2. Análisis de los Datos

El dataset contiene frases en inglés que expresan emociones y estados personales. El objetivo es clasificar cada frase en una de cinco categorías emocionales representadas por la variable *label* (0–4).

El dataset tiene 1 variable de entrada (text) y 1 variable objetivo (label).

2.1 Exploración inicial de los datos

- Número de registros: 416,810.
- Número de variables: 2.
- Tipo de datos: texto y numérica.

2.2 Observaciones iniciales:

- El contenido del texto es subjetivo, centrado en emociones y experiencias personales. Hay columnas donde la mayoría de los valores son

- Las etiquetas están desbalanceadas, con la clase 4 muy poco representada.

	count	percent
label		
1	141067	33.84
0	121187	29.07
3	57317	13.75
4	47712	11.45
2	34554	8.29
5	14972	3.59

Total muestras: 416809

Clase mayoritaria (%): 33.84 | Clase minoritaria (%): 3.59

Ratio mayoritaria/minoritaria: 9.42

- Los textos tienen diferentes longitudes, lo cual implica la necesidad de normalización (preprocesamiento).
- El dataset es relativamente pequeño, lo que puede limitar la capacidad de generalización de un modelo de aprendizaje automático.

2.3 Limpieza de datos

- Desbalanceo de valores:
 - Se detectaron categorías que estaban desbalanceadas, en este caso que tenían muy poca información, siendo la 2 y la 5 representando un 8% y 3% respectivamente, por lo que se decidieron eliminar dejando solamente 4 clases

```
df = df[~df['label'].isin([2, 5])]
df
```

	text	label
0	i feel awful about it too because it s my job ...	0
1	im alone i feel awful	0
2	ive probably mentioned this before but i reall...	1
3	i was feeling a little low few days back	0
6	i am one of those people who feels like going ...	1

- En este caso las variables quedaron recorridas en el identificador del label, por lo que se decidió hacer un reordenamiento de los identificadores para que no queden saltados en los números.

```
unique_labels = sorted(df['label'].unique())
mapping = {old: new for new, old in enumerate(unique_labels)}
df['label'] = df['label'].map(mapping)
df
```

	text	label
0	i feel awful about it too because it s my job ...	0
1	im alone i feel awful	0
2	ive probably mentioned this before but i reall...	1
3	i was feeling a little low few days back	0
6	i am one of those people who feels like going ...	1
...	...	...
416804	that was what i felt when i was finally accept...	1
416805	i take every day as it comes i m just focussin...	3
416806	i just suddenly feel that everything was fake	0
416807	im feeling more eager than ever to claw back w...	1
416808	i give you plenty of attention even when i fee...	0

367283 rows x 2 columns

- Con teste cambio quedaron las variables balanceadas eliminado ese principal problema a la hora de entrenar.

	count	percent
label		
1	141067	38.41
0	121187	33.00
2	57317	15.61
3	47712	12.99

2.4 Diccionario de las categorías

En este caso para tener las categorías identificadas en un inicio se hizo un diccionario de que representa cada numero a las emociones, para de esta forma tenerlas idénticas, quedando

1 → sadness

2 → joy

3 → anger

4 → fear

```
class_names = ["sadness", "joy", "anger", "fear"]

class_to_index = {name: i for i, name in enumerate(class_names)}
index_to_class = {i: name for i, name in enumerate(class_names)}

print(class_to_index)

{'sadness': 0, 'joy': 1, 'anger': 2, 'fear': 3}
```

2.5 División en entrenamiento y prueba

Primero el dataset se separó en:

- 70% → conjunto de entrenamiento (*train_df*)
- 30% → conjunto temporal (*temp_df*)

Luego el 30% restante se dividió en partes iguales:

- 50% → validación (*val_df*)
- 50% → prueba (*test_df*)

Lo cual equivale a:

- 15% del total → validación
- 15% del total → prueba

Manteniendo la proporción original de clases:

Train: 257098, Validation: 55092, Test: 55093

Distribución de clases:

label

0 0.330

1 0.384

2 0.156

3 0.130

Name: proportion, dtype: float64

label

0 0.330

1 0.384

2 0.156

3 0.130

Name: proportion, dtype: float64

label

0 0.330

1 0.384

2 0.156

3 0.130

Name: proportion, dtype: float64

3. Vectorización del texto

3.1. Definición de los hiperparámetros de texto

Se fijó un tamaño máximo de secuencia de 100 tokens. Todo texto más largo se trunca y todo texto más corto se rellena hasta 100. Además, se limitó el vocabulario a las 10 000 palabras más frecuentes; cualquier palabra fuera de ese top se mapea al token fuera de vocabulario.

3.2. Definición de los hiperparámetros de texto

Se configuró una capa de TextVectorization para que devuelva índices enteros. Esta capa se encargará de tokenizar, normalizar tanto minúsculas, así como separación por espacios, mapear a índices del vocabulario y aplicar padding hasta la longitud máxima.

3.3. Construcción de datasets

Del dataframe de entrenamiento, validación y prueba se extrajo la columna text y la columna label para tener arrays limpias de entrada y salida.

Con esos arrays se crearon datasets para entrenamiento, validación y prueba, agrupando los ejemplos en batches de 128 para procesarlos de forma eficiente en GPU/CPU.

3.4. Construcción de datasets

Se generó una vista del dataset que solo contiene los textos de entrenamiento, sobre esa vista se ejecutó adapt, para que la capa aprenda el vocabulario y las frecuencias a partir exclusivamente del conjunto de entrenamiento, también se adaptó una capa binaria a modo de referencia,

aunque el flujo final utiliza la vectorización entera secuencial.

Se definió una función que, dado un texto y su etiqueta, expande la dimensión del texto y aplica la capa de vectorización entera, devolviendo la secuencia de índices junto con la etiqueta correspondiente.

Por último, se transformaron los datasets de entrenamiento, validación y prueba mapeando la función anterior, el resultado son datasets donde cada muestra es una secuencia de 100 enteros y su etiqueta.

3.3. Optimización.

Se aplicaron cache y prefetch con AUTOTUNE a los datasets vectorizados, reduciendo la latencia de E/S y solapando la preparación de datos con el entrenamiento del modelo para acelerar el throughput.

4. Entrenamiento del modelo

El modelo recibe como entrada una secuencia de enteros donde cada entero representa un token del vocabulario la secuencia tiene longitud fija donde el tamaño del vocabulario está limitado a 10 000 palabras.

4.1. Capa de Embedding

La primera capa del modelo es un embedding layer:

- Convierte cada número entero (ID de palabra) en un vector denso.
- En vez de representar palabras como enteros individuales, las transforma en vectores que capturan significado.
- Cada palabra termina representada por un vector continuo de cierta dimensión, en este caso 64.

Esto es útil para que las palabras similares quedan representadas por vectores similares, mejora la capacidad del modelo para aprender patrones y relaciones semánticas y además Reduce la dispersión pues en lugar de datos dispersos, se obtiene una representación eficiente.

4.2. Capa Bidirectional + LSTM

Procesa la secuencia palabra por palabra, pero en ambas direcciones:

- LSTM hacia adelante (del primer token al último)
- LSTM hacia atrás (del último token al primero)

Luego concatena ambas salidas, de esta forma captura el contexto anterior y posterior, entiende dependencias largas

en la secuencia y es capaz de aprender estructura del lenguaje.

Se decidió usar 64 unidades porque es un buen punto medio entre capacidad y eficiencia siendo unidades suficientes para captar patrones y no demasiado grande como para sobreajustar con poco texto.

4.3. Capa Dense de 64 neuronas

Transforma la información extraída por la LSTM en patrones más abstractos.

Se usa la activación ReLU el cual ayuda a capturar relaciones no lineales y evita saturación

4.4. Capa Dense de 64 neuronas

Produce una distribución de probabilidades entre las 4 clases, esperando la siguiente salida:

[0.02, 0.10, 0.78, 0.10]

Donde la clase predicha sería la clase 2 siendo probabilidad más alta.

En resumen, el modelo es una red neuronal secuencial diseñada para clasificar texto en 4 categorías. Primero convierte cada palabra en un vector continuo mediante una capa Embedding. Luego utiliza una capa Bidirectional LSTM, que analiza la secuencia en ambas direcciones para capturar mejor el contexto. Después, una capa Dense de 64 neuronas refina la representación aprendida.

Finalmente, una capa Dense con softmax produce las probabilidades de cada una de las 4 clases.

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 100, 64)	640,000
bidirectional (Bidirectional)	(None, 128)	66,048
dense (Dense)	(None, 64)	8,256
dense_1 (Dense)	(None, 4)	260

Total params: 2,143,694 (8.18 MB)
Trainable params: 714,564 (2.73 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,429,130 (5.45 MB)

4.5. Entrenamiento

Epoch 1/10	35s 13ms/step - accuracy: 0.8363 - loss: 0.3839 - val_accuracy: 0.9721 - val_loss: 0.0587
Epoch 2/10	25s 13ms/step - accuracy: 0.9742 - loss: 0.0495 - val_accuracy: 0.9727 - val_loss: 0.0486
Epoch 3/10	25s 13ms/step - accuracy: 0.9759 - loss: 0.0423 - val_accuracy: 0.9727 - val_loss: 0.0505
Epoch 4/10	25s 13ms/step - accuracy: 0.9768 - loss: 0.0388 - val_accuracy: 0.9727 - val_loss: 0.0542
Epoch 5/10	26s 13ms/step - accuracy: 0.9779 - loss: 0.0365 - val_accuracy: 0.9727 - val_loss: 0.0502
Epoch 6/10	27s 13ms/step - accuracy: 0.9787 - loss: 0.0338 - val_accuracy: 0.9721 - val_loss: 0.0526
Epoch 7/10	26s 13ms/step - accuracy: 0.9796 - loss: 0.0322 - val_accuracy: 0.9734 - val_loss: 0.0545
Epoch 8/10	28s 14ms/step - accuracy: 0.9803 - loss: 0.0313 - val_accuracy: 0.9724 - val_loss: 0.0576
Epoch 9/10	25s 12ms/step - accuracy: 0.9815 - loss: 0.0299 - val_accuracy: 0.9719 - val_loss: 0.0593
Epoch 10/10	25s 12ms/step - accuracy: 0.9819 - loss: 0.0295 - val_accuracy: 0.9721 - val_loss: 0.0607

El modelo se entrenó durante 10 épocas usando el conjunto de entrenamiento y validación. A lo largo del proceso, la métrica principal de interés fue la accuracy y la loss.

En la primera época, el modelo ya alcanza:

- 83.6% de accuracy en entrenamiento
- 97.2% de accuracy en validación

Esto indica que el modelo aprende rápidamente patrones relevantes del texto, durante las siguientes épocas, el accuracy sigue aumentando suavemente:

- Final del entrenamiento: 98.19%

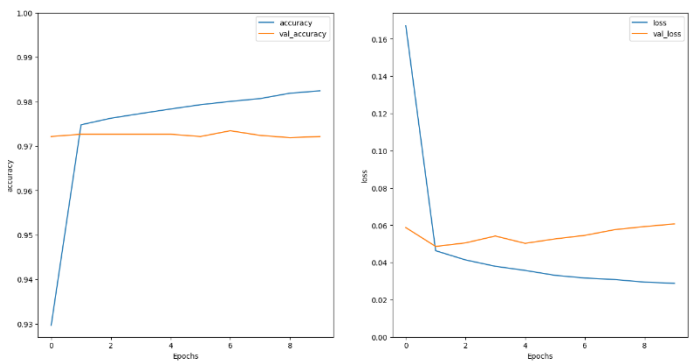
La pérdida disminuye consistentemente, lo que indica estabilidad en el aprendizaje.

A lo largo de todo el entrenamiento, la precisión en validación oscila entre 97.1% y 97.3%, mostrando generalización sólida, comportamiento consistente y ausencia de fluctuaciones peligrosas

A partir de las épocas 4–5, el modelo mejora en entrenamiento, pero el rendimiento en validación ya no aumenta.

Esto es una señal típica de que el modelo ya ha aprendido todo lo que puede aprender del conjunto de entrenamiento.

Las gráficas de cómo se entrenan son:



5. Pruebas

Después de entrenar y validar el modelo, se realizó la evaluación final utilizando el conjunto de prueba (test_dataset). Este conjunto contiene ejemplos que el modelo nunca ha visto durante el entrenamiento ni durante la validación.

Esto permite medir la verdadera capacidad del modelo para generalizar a nuevos datos.

Los resultados fueron:

- Test Accuracy: 97.13%
- Test Loss: 0.0687

Los valores coinciden con lo esperado según el comportamiento del modelo durante validación.

Como podemos ver:

- El modelo muestra un rendimiento excelente.
- No hay caídas significativas respecto al conjunto de validación.
- Se confirma que la arquitectura elegida (Embedding + BiLSTM + Dense) es adecuada para esta tarea.
- Los resultados son estables y confiables.

5.1. Pruebas personalizadas

Para no quedarse con lo realizado se decidió hacer oraciones personalizadas para ver si es que responde bien el código siendo:

```
texts = [  
    # anger  
    "I feel frustrated with the project",  
    "This is infuriating, nothing works as expected",  
  
    # joy  
    "I'm really happy with our progress",  
    "Today was amazing, everything clicked",  
  
    # sadness  
    "I feel disappointed by the outcome",  
    "Today I just feel empty and tired",  
  
    # fear  
    "I'm worried we might miss the deadline",  
    "I feel anxious about presenting tomorrow",  
]
```

Donde modelo recibió 8 frases clasificadas en 4 emociones:

- anger
- joy
- sadness
- fear

Dando de resultado:

	text	sadness	joy	anger	fear	predicted	confidence
0	I feel frustrated with the project	3.533618e-07	2.092713e-07	9.999929e-01	6.463360e-06	anger	0.999993
1	This is infuriating, nothing works as expected	1.005905e-05	3.604271e-03	4.936927e-02	9.470164e-01	fear	0.947016
2	I'm really happy with our progress	1.635930e-03	9.835390e-01	1.047417e-02	4.350950e-03	joy	0.983539
3	Today was amazing, everything clicked	1.234138e-03	9.862754e-01	1.214543e-02	3.449385e-04	joy	0.986275
4	I feel disappointed by the outcome	9.999999e-01	1.859356e-10	7.584397e-10	4.259320e-12	sadness	1.000000
5	Today I just feel empty and tired	9.999967e-01	3.164711e-06	1.891683e-08	1.018551e-09	sadness	0.999997
6	I'm worried we might miss the deadline	4.135179e-01	7.941324e-02	4.247754e-01	8.229350e-02	anger	0.424775
7	I feel anxious about presenting tomorrow	9.215233e-06	2.171093e-08	5.524936e-06	9.999852e-01	fear	0.999985

Para cada frase se muestran:

- Las probabilidades asignadas a cada emoción,
- La emoción predicha,
- El nivel de confianza.

El modelo demuestra un comportamiento coherente con el significado emocional de los textos.

6. Conclusión General

Este proyecto implementó un flujo completo de procesamiento de lenguaje natural (NLP) orientado a la clasificación emocional del texto. Desde la preparación de datos hasta la evaluación final, el modelo demostró un rendimiento sólido y consistente.

El modelo emplead fue capaz de capturar tanto el significado semántico como el contexto temporal de las frases. Su desempeño durante el entrenamiento fue excelente, con una rápida convergencia y estabilidad en la validación, sin señales significativas de sobreajuste.

Los resultados en testing confirmaron la robustez del modelo, alcanzando una precisión superior al 97%, muy cercana a la obtenida en validación. Esto demuestra una correcta capacidad para generalizar a datos no vistos.

Finalmente, al evaluar ejemplos personalizados, el modelo mostró interpretaciones coherentes y altas confianzas para la mayoría de los textos, capturando adecuadamente distintas emociones humanas como enojo, alegría, tristeza y miedo. Solo algunos casos ambiguos mostraron incertidumbre, reflejando la complejidad natural del lenguaje emocional.